

1. Determine your system configuration:

- a. **Specify what eos system you are working on**
Using the EOS RDP
- b. the total amount of physical memory (KB) on your system
Using the free -kilo command the total amount of memory is: 16,442,728
- c. The current amount of free memory (KB)
The amount of free memory on my system is: 5976518
- d. The total amount of swap space
We got : 4294926 of total swap space

2. Examine and observe the memory demand of an executing process

- **Study the Sample Program. What is your estimate of the approximate memory demand of the Sample Program?**

$2,048 * 2,048 * 4 = 16,777,216$ bytes, so 16.8 MB

- **In another window, execute the Sample Program**
 - a. **approximately how much does the amount of free (idle) memory change?**
27.4 MB
 - b. **Considering your estimated memory demand of the Sample Program (question 2a), explain why the observed change is an expected result.**

This is most likely due to the fact we're on an EOS machine where lots of stuff is running and also that we are only looking at a snapshot of the program running instead of the entire time it was running.

3. Examine the effect of memory access patterns:

- a. **obtain basic statistics**
 - **what is the size of a page in Linux?**
4,096 bytes
- b. **How long does the program take to run?**
 - 1.98 seconds
- **Change the memory access statement in the Sample Program. Then re-compile and re-run the program, collecting statistics on execution time.**
 - a. **Precisely, how does this change alter the program's memory access pattern (i.e. what memory objects get "touched", and in what order)? A diagram will help here.**

In the original sample program 1, we see the memory access pattern starts from the very first spot in the free memory all the way to the end, incrementing by 1. If we were to refer to it as an array for example, it would touch index 0, and then go all the way up to the last index of the array. In the diagram below, the numbers are the order that the memory objects get touched. We can see that it starts at the first index of the malloced space, and increments by 1 until it gets to the very end of the malloced space.

1 we see that it touches address 1, and then repeats this cycle of touching addresses increases by 2048 until we get to point $x + y$. We see that once again it touches an early address, and then reaches out by 2048, and repeats.

b. How does this change affect the program's (user) execution time?

It increases the execution time from 1.98 seconds to 6.19 seconds

c. Precisely, why does the change have the observed effect (your answer must incorporate an important concept related to paging and virtual memory)

This change causes an effect because processes are broken into logical address spaces in virtual memory of certain sizes depending on the computer being used. Since we have a malloced space of such a massive size, it doesn't continuously allocate one humongous segment all back to back. Some may be stored at different physical addresses, but because of virtual memory, it allows us to act as though it is one huge malloced space that is stored continuously. However only so much of our process can fit on a frame. So the increased time is due to this virtual memory and paging. Paging adds overhead of now finding the correct page needed for this part in the process. Due to this increased overhead, there's an increased amount of time which is why there's the observed effect that occurs. However, it is good because it allows us to have processes that are bigger than a page size like this one, and still run them.

4. Examine the use of virtual memory

- Change the program back to its original state
- Change the Loop value to 1

Adjust the COEFFICIENT parameter in the Sample Program to a value that causes the memory demand of the program to exceed the total amount of physical memory on your machine (as determined in question 1 above).

a.

For the value of COEFFICIENT, I used 67. Since we are `malloc(dim * dim * 4)` I want a value that gets me over the amount of physical memory we found in part 1 which is 16,442,728. The dim variable is equal to $\text{COEFFICIENT} * \text{KB}$ so to `malloc()` more space than what we have above. The value has to come from $\text{COEFFICIENT}^2 * 1024^2 * 4$. Then to find the COEFFICIENT in KB, take that equation and divide 1024, and you end up with $\text{COEFFICIENT}^2 * 4096 > 16,442,728$. Take the square root of $(16,442,728 / 4096)$ and you get something around 64. Then I added 3 more just be on the safe side.

```

procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
r b  swpd  free  buff  cache  si  so    bi    bo    in  cs  us  sy  id  wa  st  gu
2 0  1357276 14423072 14900 471788 0 1 7 203 796 1 1 0 98 0 0 0
0 0  1357020 14424908 14900 471856 16 0 16 8 931 1601 1 0 99 0 0 0
1 0  1356764 14425812 14900 471852 52 0 52 0 1378 2764 3 0 96 0 0 0
1 0  1356764 14340524 14900 471856 200 0 200 0 1390 1944 3 1 96 0 0 0
1 0  1356764 13081076 14908 475848 56 0 4044 116 1506 970 6 3 91 0 0 0
1 0  1355740 11825660 14908 475848 256 0 256 0 1900 2003 7 2 91 0 0 0
1 0  1355740 10573848 14908 475848 4 0 4 132 1480 887 6 2 91 0 0 0
1 0  1355484 9316656 14908 475952 252 0 356 0 1840 1838 7 2 91 0 0 0
1 0  1355484 8069888 14916 475952 48 0 56 0 1358 748 6 2 91 0 0 0
1 0  1355484 6784612 14924 475896 252 0 252 56 1997 1978 7 3 90 0 0 0
1 0  1355484 5529492 14924 477608 480 0 2192 0 1578 1147 6 2 91 0 0 0
1 1  1355228 4292344 14924 477592 136 0 136 8 2770 3439 7 3 90 0 0 0
4 0  1355228 3045716 14924 477592 12 0 12 0 1435 805 6 2 91 0 0 0
1 0  1355228 1768840 14924 477592 56 0 56 0 2098 2051 7 2 90 0 0 0
1 0  1355228 509136 14924 477592 4 0 4 0 1361 874 6 2 92 0 0 0
2 0  2060252 184220 7440 462580 184 705052 184 705132 173611 2175 7 6 86 1 0 0
2 1  3176668 196228 10064 502016 1668 1118980 52276 1119120 291239 5397 7 7 82 4 0 0
2 0  4150628 161104 10376 515244 408 1037888 17968 1037892 285881 8348 8 7 82 2 0 0
0 0  1397748 14780912 2756 244820 788 217684 109008 217684 28182 4658 3 9 79 9 0 0
1 0  1397748 14733524 6092 268740 588 0 27912 0 2453 3594 3 1 95 1 0 0
0 0  1380084 14690140 6540 300956 20160 0 34996 0 3147 5337 2 1 96 1 0 0
procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
r b  swpd  free  buff  cache  si  so    bi    bo    in  cs  us  sy  id  wa  st  gu
0 0  1380084 14682148 6832 310604 256 0 10160 88 921 1338 1 0 99 0 0 0
0 0  1379828 14636576 7964 329148 944 0 17396 0 2336 3649 3 2 94 1 0 0
^C

```

b.

The Highlighted circle shows what happens to the memory when the program runs with the COEFFICEINT AT 67 the free memory drops significantly. We can also see the memory buffer changes. You can also tell from the highlighted lines that swaps in increased, swaps out increased, Blocks in increased, blocks out increased, interrupts increases, context switches increased, and user CPU time also increased.

c.

The operating system makes adjustments to memory as it swaps out pages and tries to make room for all of the data needed. We did notice that the execution time did increase as seen in the sy and us fields. The time the program runs increases because we are doing alot of page swapping in and out.